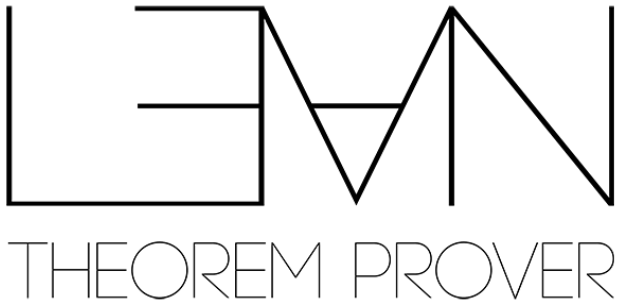
Formal Theorem Proving in Lean

William G. Underwood

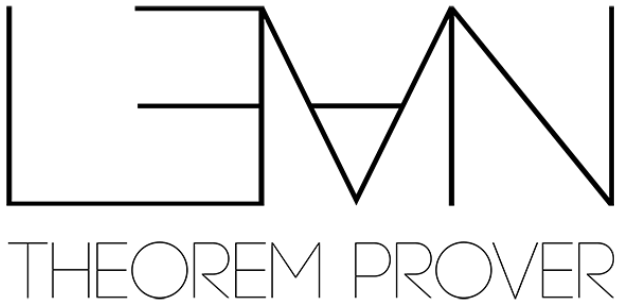
2nd November, 2026

Department of Statistics, University of Warwick

What is Lean?

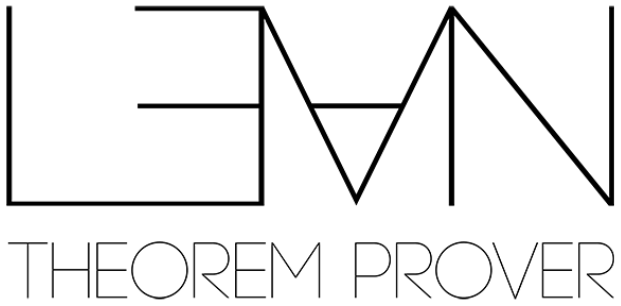


What is Lean?



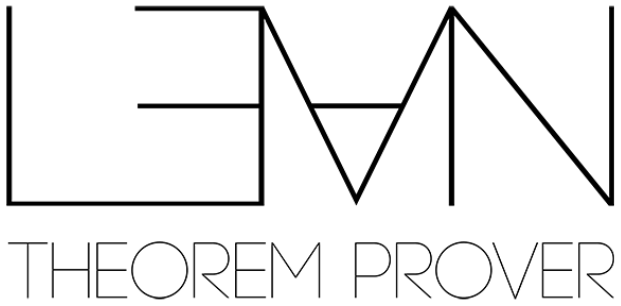
- Functional programming language and theorem prover

What is Lean?



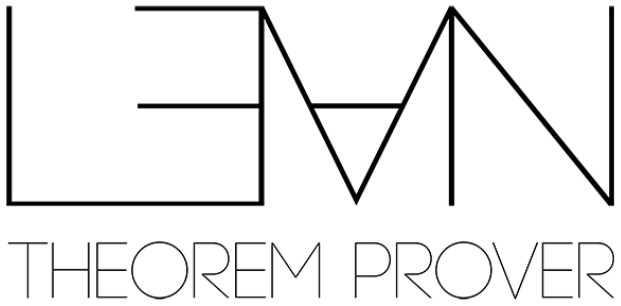
- Functional programming language and theorem prover
- Designed for formal verification tasks

What is Lean?



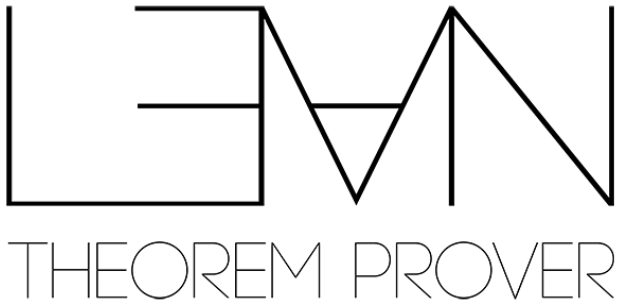
- Functional programming language and theorem prover
- Designed for formal verification tasks
- Based on dependent type theory

What is Lean?



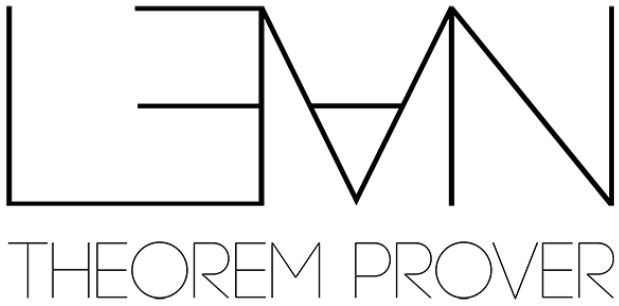
- Functional programming language and theorem prover
- Designed for formal verification tasks
- Based on dependent type theory
- Open-source, initially developed by Microsoft

What is Lean?



- Functional programming language and theorem prover
- Designed for formal verification tasks
- Based on dependent type theory
- Open-source, initially developed by Microsoft
- First launched in 2013; Lean 4 released in 2021

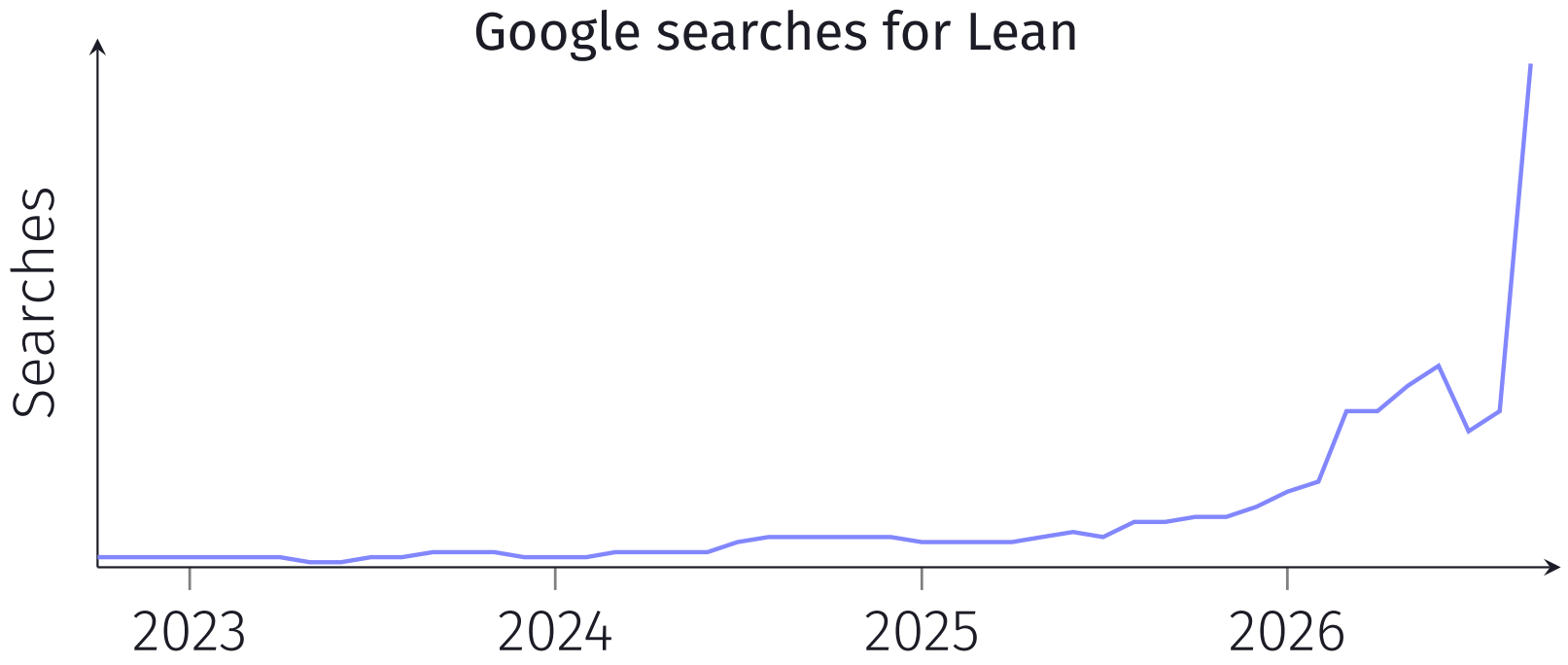
What is Lean?



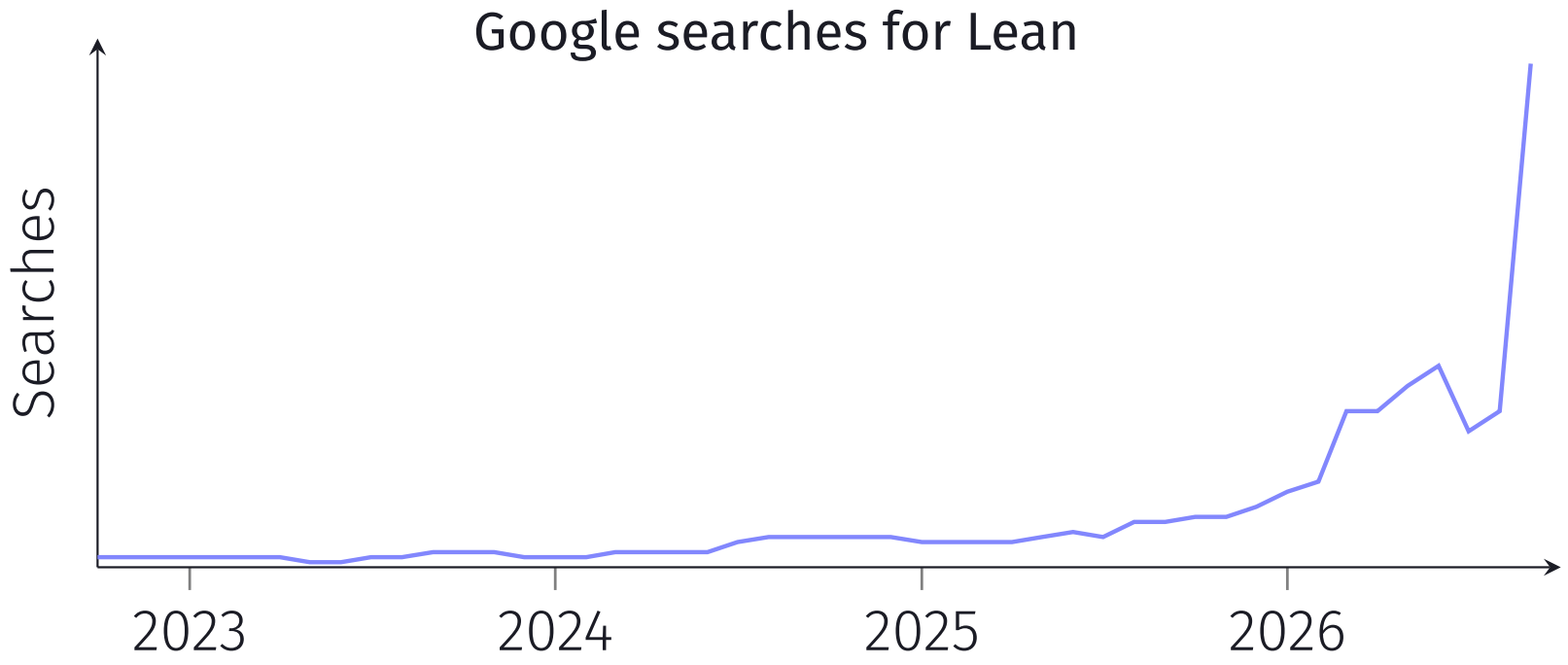
- Functional programming language and theorem prover
- Designed for formal verification tasks
- Based on dependent type theory
- Open-source, initially developed by Microsoft
- First launched in 2013; Lean 4 released in 2021
- Try it in your browser (no installation needed) at:

`https://live.lean-lang.org`

Lean and AI-generated mathematics

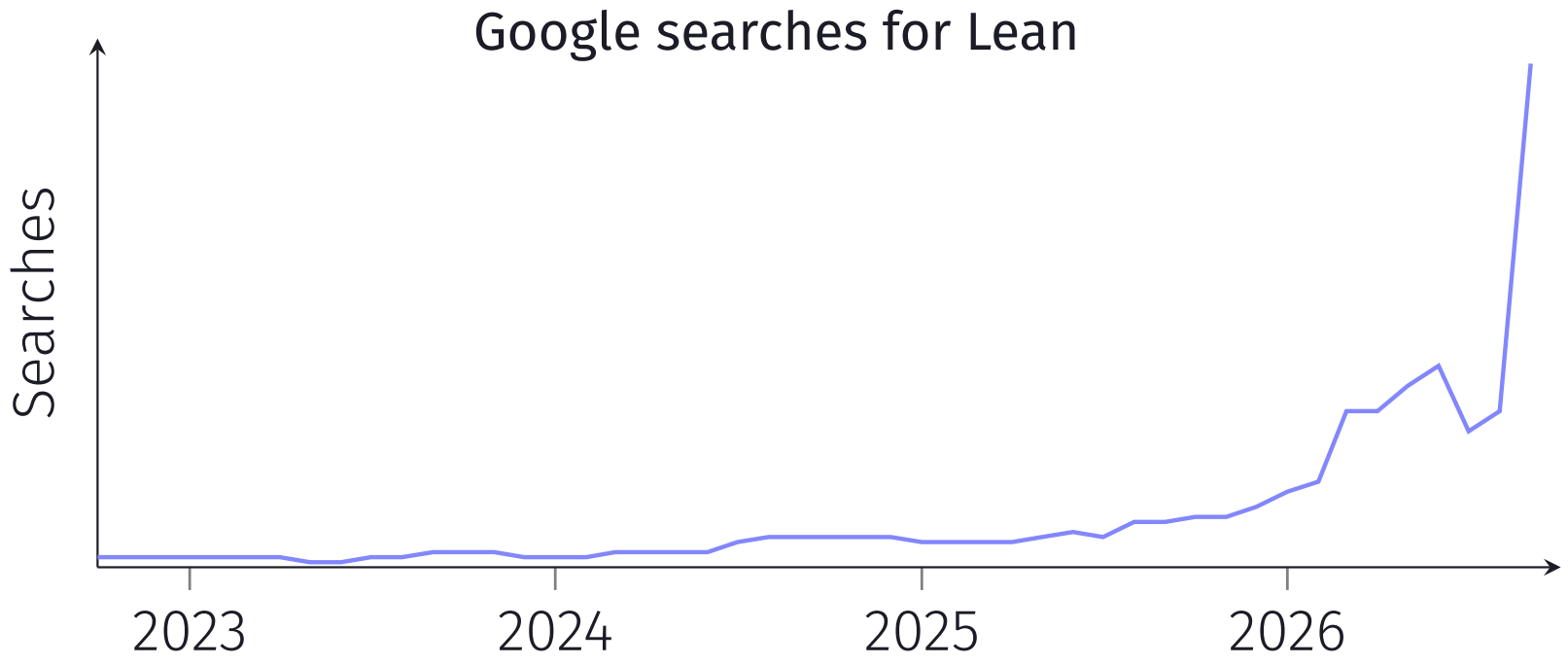


Lean and AI-generated mathematics



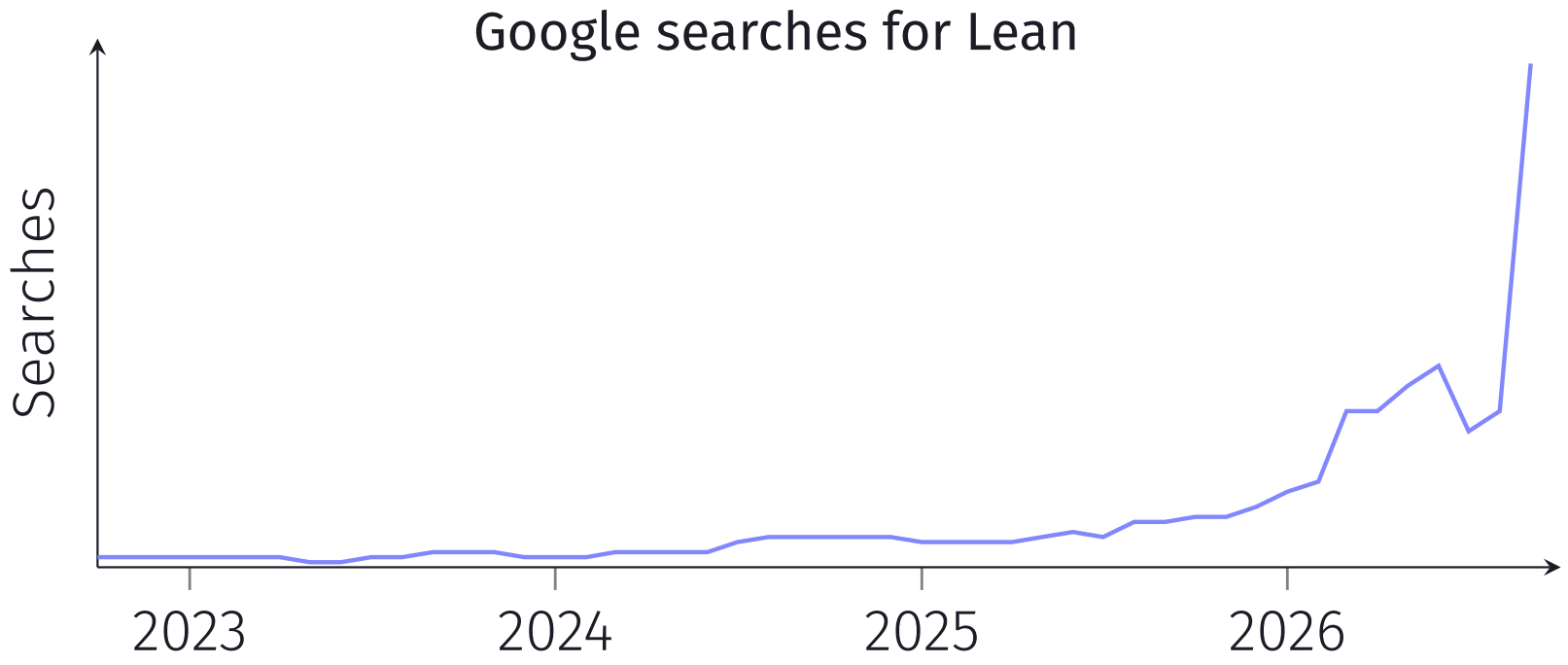
- Jul 2024: DeepMind's AlphaProof solves IMO problems.

Lean and AI-generated mathematics



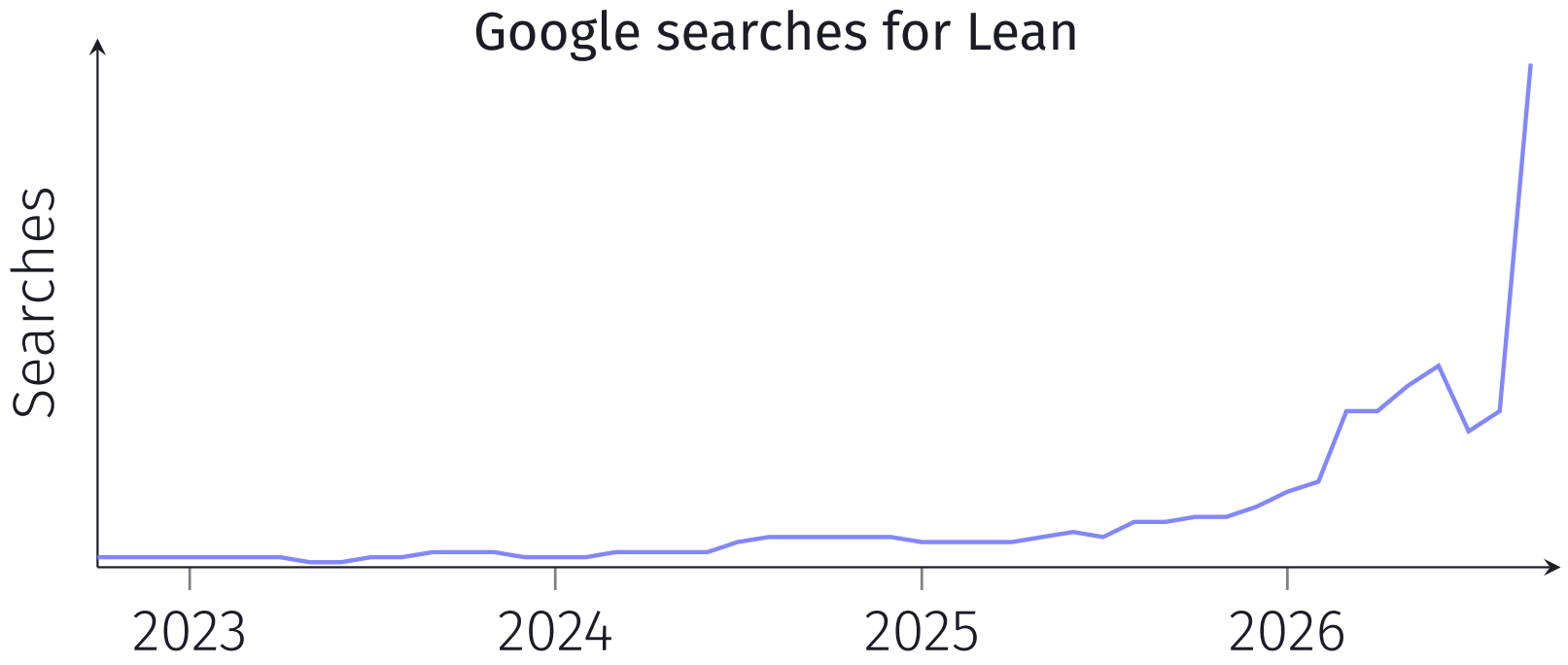
- Jul 2024: DeepMind's AlphaProof solves IMO problems.
- Jan 2026: OpenAI resolves an Erdős problem.

Lean and AI-generated mathematics



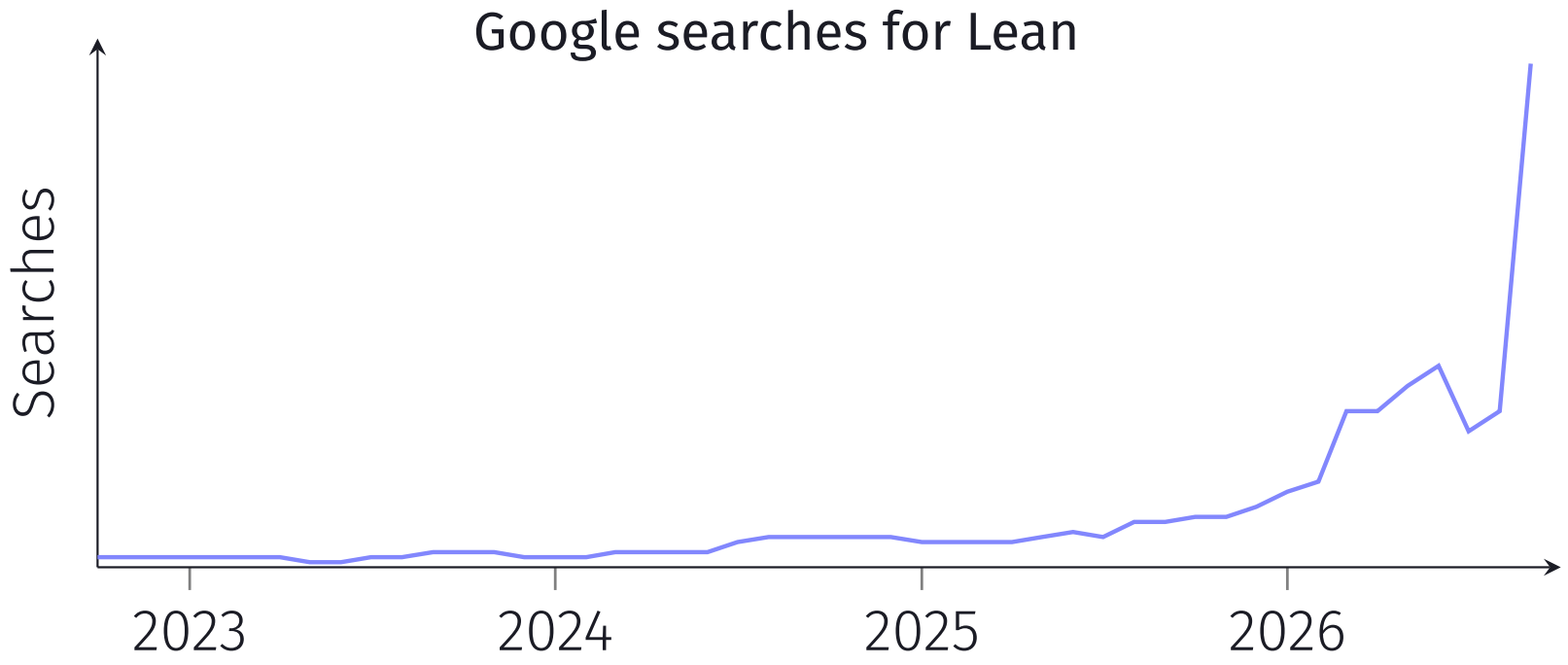
- Jul 2024: DeepMind's AlphaProof solves IMO problems.
- Jan 2026: OpenAI resolves an Erdős problem.
- May 2026: OpenAI disproves the unit distance conjecture.

Lean and AI-generated mathematics



- Jul 2024: DeepMind's AlphaProof solves IMO problems.
- Jan 2026: OpenAI resolves an Erdős problem.
- May 2026: OpenAI disproves the unit distance conjecture.
- Sep 2026: OpenAI resolves Navier–Stokes.

Lean and AI-generated mathematics



- Jul 2024: DeepMind's AlphaProof solves IMO problems.
- Jan 2026: OpenAI resolves an Erdős problem.
- May 2026: OpenAI disproves the unit distance conjecture.
- Sep 2026: OpenAI resolves Navier–Stokes.
- Oct 2026: OpenAI disproves the exponential time hypothesis.

The Mathlib repository

- Mathlib is a library (or package) for Lean, started in 2017.

The Mathlib repository

- Mathlib is a library (or package) for Lean, started in 2017.
- Serves as a repository of formalised mathematics.

The Mathlib repository

- Mathlib is a library (or package) for Lean, started in 2017.
- Serves as a repository of formalised mathematics.
- Now contains 200k theorems and 2.4M lines of code.

The Mathlib repository

- Mathlib is a library (or package) for Lean, started in 2017.
- Serves as a repository of formalised mathematics.
- Now contains 200k theorems and 2.4M lines of code.
- Includes measure theory and probability but not statistics.

The Mathlib repository

- Mathlib is a library (or package) for Lean, started in 2017.
- Serves as a repository of formalised mathematics.
- Now contains 200k theorems and 2.4M lines of code.
- Includes measure theory and probability but not statistics.
- E.g., the strong law of large numbers is `strong_law_ae`.

The Mathlib repository

- Mathlib is a library (or package) for Lean, started in 2017.
- Serves as a repository of formalised mathematics.
- Now contains 200k theorems and 2.4M lines of code.
- Includes measure theory and probability but not statistics.
- E.g., the strong law of large numbers is `strong_law_ae`.
- Mathlib also provides tooling to make Lean easier to use.

`github.com/leanprover-community/mathlib4`

Writing our first proofs

Writing our first proofs

- `example`: Introduce a new proposition to prove.

Writing our first proofs

- `example`: Introduce a new proposition to prove.
- `rfl`: Prove a proposition by definition.

Writing our first proofs

- `example`: Introduce a new proposition to prove.
- `rfl`: Prove a proposition by definition.
- `exact h`: Prove a proposition by applying hypothesis `h`.

Writing our first proofs

- `example`: Introduce a new proposition to prove.
- `rfl`: Prove a proposition by definition.
- `exact h`: Prove a proposition by applying hypothesis `h`.
- `rw [h]`: Rewrite the goal using hypothesis `h`.

Writing our first proofs

- `example`: Introduce a new proposition to prove.
- `rfl`: Prove a proposition by definition.
- `exact h`: Prove a proposition by applying hypothesis `h`.
- `rw [h]`: Rewrite the goal using hypothesis `h`.
- Comments can be written with: `-- here is a comment`.

Writing our first proofs

- `example`: Introduce a new proposition to prove.
- `rfl`: Prove a proposition by definition.
- `exact h`: Prove a proposition by applying hypothesis `h`.
- `rw [h]`: Rewrite the goal using hypothesis `h`.
- Comments can be written with: `-- here is a comment`.
- `#check`: Check the type of an expression.
  `#check 1` returns `1:`$\mathbb{N}$.
  `#check (1 + 1 = 2)` returns `(1 + 1 = 2):Prop`.

Writing our first proofs

- `example`: Introduce a new proposition to prove.
- `rfl`: Prove a proposition by definition.
- `exact h`: Prove a proposition by applying hypothesis `h`.
- `rw [h]`: Rewrite the goal using hypothesis `h`.
- Comments can be written with: `-- here is a comment`.
- `#check`: Check the type of an expression.
  `#check 1` returns `1:ℕ`.
  `#check (1 + 1 = 2)` returns `(1 + 1 = 2):Prop`.
- `constructor`: Transform `P and Q` into two goals, `P` and `Q`.

Writing our first proofs

- `example`: Introduce a new proposition to prove.
- `rfl`: Prove a proposition by definition.
- `exact h`: Prove a proposition by applying hypothesis `h`.
- `rw [h]`: Rewrite the goal using hypothesis `h`.
- Comments can be written with: `-- here is a comment`.
- `#check`: Check the type of an expression.
  `#check 1` returns `1:ℕ`.
  `#check (1 + 1 = 2)` returns `(1 + 1 = 2):Prop`.
- `constructor`: Transform `P and Q` into two goals, `P` and `Q`.
- `sorry`: Temporarily prove any goal (and raise a warning).

Writing our first proofs

- `example`: Introduce a new proposition to prove.
- `rfl`: Prove a proposition by definition.
- `exact h`: Prove a proposition by applying hypothesis `h`.
- `rw [h]`: Rewrite the goal using hypothesis `h`.
- Comments can be written with: `-- here is a comment`.
- `#check`: Check the type of an expression.
  `#check 1` returns `1:ℕ`.
  `#check (1 + 1 = 2)` returns `(1 + 1 = 2):Prop`.
- `constructor`: Transform `P and Q` into two goals, `P` and `Q`.
- `sorry`: Temporarily prove any goal (and raise a warning).
- `have`: Introduce a new sub-goal to prove.

Basics of type-theoretic logic

- The Curry–Howard correspondence gives an isomorphism between models for *type theory* and *logic*.

Basics of type-theoretic logic

- The Curry–Howard correspondence gives an isomorphism between models for *type theory* and *logic*.

Type theory	Logic
Type `T` Element of type `x:T`	Proposition `P` Proof of proposition `h:P`
`ℕ:Type` `n:ℕ`	`(1 + 1 = 2):Prop` `rfl:(1 + 1 = 2)`

Basics of type-theoretic logic

- The Curry–Howard correspondence gives an isomorphism between models for *type theory* and *logic*.

Type theory	Logic
Type `T` Element of type `x:T`	Proposition `P` Proof of proposition `h:P`
`ℕ:Type` `n:ℕ`	`(1 + 1 = 2):Prop` `rfl:(1 + 1 = 2)`

- The proposition `True` has a proof `trivial:True`.

Basics of type-theoretic logic

- The Curry–Howard correspondence gives an isomorphism between models for *type theory* and *logic*.

Type theory	Logic
Type `T` Element of type `x:T`	Proposition `P` Proof of proposition `h:P`
`ℕ:Type` `n:ℕ`	`(1 + 1 = 2):Prop` `rfl:(1 + 1 = 2)`

- The proposition `True` has a proof `trivial:True`.
- The proposition `False` has no proofs!

Basics of type-theoretic logic

- If P and Q are propositions, then a proof of $(P \Longrightarrow Q)$ is a function mapping proofs of P to proofs of Q.

Basics of type-theoretic logic

- If `P` and `Q` are propositions, then a proof of `(P ⟹ Q)` is a function mapping proofs of `P` to proofs of `Q`.
- A proof of `(P and Q)` is a pair of proofs: `(hP, hQ):(P and Q)` where `hP:P` and `hQ:Q`.

Basics of type-theoretic logic

- If `P` and `Q` are propositions, then a proof of `(P ⟹ Q)` is a function mapping proofs of `P` to proofs of `Q`.
- A proof of `(P and Q)` is a pair of proofs: `(hP, hQ):(P and Q)` where `hP:P` and `hQ:Q`.
- A proof of `(not P)` maps proofs of `P` to proofs of `False`.

Basics of type-theoretic logic

- If `P` and `Q` are propositions, then a proof of `(P ⟹ Q)` is a function mapping proofs of `P` to proofs of `Q`.
- A proof of `(P and Q)` is a pair of proofs: `(hP, hQ):(P and Q)` where `hP:P` and `hQ:Q`.
- A proof of `(not P)` maps proofs of `P` to proofs of `False`.
- `(P or Q)` is more complicated: we may prove `(P or Q)` without a proof either for `P` or for `Q`. For example:

$$\exists p, q \in \mathbb{R} \setminus \mathbb{Q} : p^q \in \mathbb{Q}.$$

Proof. Consider $x \coloneqq \sqrt{2}^{\sqrt{2}}$ and $x^{\sqrt{2}}$. □

A more interesting hand-written proof

Theorem (Infinitude of the primes)

For all $n \in \mathbb{N}$, there exists $p \in \mathbb{N}$ such that $p > n$ and p is prime.

A more interesting hand-written proof

Theorem (Infinitude of the primes)

For all $n \in \mathbb{N}$, there exists $p \in \mathbb{N}$ such that $p > n$ and p is prime.

Proof

Take $n \in \mathbb{N}$, write $m := n! + 1$, and let p be a prime factor of m.

A more interesting hand-written proof

Theorem (Infinitude of the primes)

For all $n \in \mathbb{N}$, there exists $p \in \mathbb{N}$ such that $p > n$ and p is prime.

Proof

Take $n \in \mathbb{N}$, write $m := n! + 1$, and let p be a prime factor of m.

- Note that $m \neq 1$ because $n! \neq 0$.

A more interesting hand-written proof

Theorem (Infinitude of the primes)

For all $n \in \mathbb{N}$, there exists $p \in \mathbb{N}$ such that $p > n$ and p is prime.

Proof

Take $n \in \mathbb{N}$, write $m := n! + 1$, and let p be a prime factor of m.

- Note that $m \neq 1$ because $n! \neq 0$.
- Therefore, p is prime.

A more interesting hand-written proof

Theorem (Infinitude of the primes)

For all $n \in \mathbb{N}$, there exists $p \in \mathbb{N}$ such that $p > n$ and p is prime.

Proof

Take $n \in \mathbb{N}$, write $m := n! + 1$, and let p be a prime factor of m.

- Note that $m \neq 1$ because $n! \neq 0$.
- Therefore, p is prime.
- Suppose for contradiction that we do not have $p > n$.

A more interesting hand-written proof

Theorem (Infinitude of the primes)

For all $n \in \mathbb{N}$, there exists $p \in \mathbb{N}$ such that $p > n$ and p is prime.

Proof

Take $n \in \mathbb{N}$, write $m := n! + 1$, and let p be a prime factor of m.

- Note that $m \neq 1$ because $n! \neq 0$.
- Therefore, p is prime.
- Suppose for contradiction that we do not have $p > n$.
- Then $p \leq n$.

A more interesting hand-written proof

Theorem (Infinitude of the primes)

For all $n \in \mathbb{N}$, there exists $p \in \mathbb{N}$ such that $p > n$ and p is prime.

Proof

Take $n \in \mathbb{N}$, write $m := n! + 1$, and let p be a prime factor of m.

- Note that $m \neq 1$ because $n! \neq 0$.
- Therefore, p is prime.
- Suppose for contradiction that we do not have $p > n$.
- Then $p \leq n$.
- Note that $p > 0$ because p is a factor of m.

A more interesting hand-written proof

Theorem (Infinitude of the primes)

For all $n \in \mathbb{N}$, there exists $p \in \mathbb{N}$ such that $p > n$ and p is prime.

Proof

Take $n \in \mathbb{N}$, write $m := n! + 1$, and let p be a prime factor of m.

- Note that $m \neq 1$ because $n! \neq 0$.
- Therefore, p is prime.
- Suppose for contradiction that we do not have $p > n$.
- Then $p \leq n$.
- Note that $p > 0$ because p is a factor of m.
- We have $p \mid m$ by definition of p.

A more interesting hand-written proof

Theorem (Infinitude of the primes)

For all $n \in \mathbb{N}$, there exists $p \in \mathbb{N}$ such that $p > n$ and p is prime.

Proof

Take $n \in \mathbb{N}$, write $m := n! + 1$, and let p be a prime factor of m.

- Note that $m \neq 1$ because $n! \neq 0$.
- Therefore, p is prime.
- Suppose for contradiction that we do not have $p > n$.
- Then $p \leq n$.
- Note that $p > 0$ because p is a factor of m.
- We have $p \mid m$ by definition of p.
- Also, $p \mid n!$ because $p \leq n$ by assumption and $p > 0$.

A more interesting hand-written proof

Theorem (Infinitude of the primes)

For all $n \in \mathbb{N}$, there exists $p \in \mathbb{N}$ such that $p > n$ and p is prime.

Proof

Take $n \in \mathbb{N}$, write $m := n! + 1$, and let p be a prime factor of m.

- Note that $m \neq 1$ because $n! \neq 0$.
- Therefore, p is prime.
- Suppose for contradiction that we do not have $p > n$.
- Then $p \leq n$.
- Note that $p > 0$ because p is a factor of m.
- We have $p \mid m$ by definition of p.
- Also, $p \mid n!$ because $p \leq n$ by assumption and $p > 0$.
- So $p \mid 1$ by modular arithmetic.

A more interesting hand-written proof

Theorem (Infinitude of the primes)

For all $n \in \mathbb{N}$, there exists $p \in \mathbb{N}$ such that $p > n$ and p is prime.

Proof

Take $n \in \mathbb{N}$, write $m := n! + 1$, and let p be a prime factor of m.

- Note that $m \neq 1$ because $n! \neq 0$.
- Therefore, p is prime.
- Suppose for contradiction that we do not have $p > n$.
- Then $p \leq n$.
- Note that $p > 0$ because p is a factor of m.
- We have $p \mid m$ by definition of p.
- Also, $p \mid n!$ because $p \leq n$ by assumption and $p > 0$.
- So $p \mid 1$ by modular arithmetic.
- Thus, $p = 1$.

A more interesting hand-written proof

Theorem (Infinitude of the primes)

For all $n \in \mathbb{N}$, there exists $p \in \mathbb{N}$ such that $p > n$ and p is prime.

Proof

Take $n \in \mathbb{N}$, write $m := n! + 1$, and let p be a prime factor of m.

- Note that $m \neq 1$ because $n! \neq 0$.
- Therefore, p is prime.
- Suppose for contradiction that we do not have $p > n$.
- Then $p \leq n$.
- Note that $p > 0$ because p is a factor of m.
- We have $p \mid m$ by definition of p.
- Also, $p \mid n!$ because $p \leq n$ by assumption and $p > 0$.
- So $p \mid 1$ by modular arithmetic.
- Thus, $p = 1$.
- But 1 is not prime ※. □

Writing an AI-assisted proof

Theorem

Let $X_1, \ldots, X_n$ be independent and identically distributed random variables with mean $\mathbb{E}(X_1) = 0$ and variance $\mathbb{E}(X_1^2) = \sigma^2 < \infty$. Then, for all $\varepsilon > 0$, we have

$$\mathbb{P}\left(\left|\frac{1}{n}\sum_{i=1}^{n} X_i\right| > \varepsilon\right) \leq \frac{\sigma^2}{n\varepsilon^2}.$$

Writing an AI-assisted proof

Theorem

Let $X_1, \ldots, X_n$ be independent and identically distributed random variables with mean $\mathbb{E}(X_1) = 0$ and variance $\mathbb{E}(X_1^2) = \sigma^2 < \infty$. Then, for all $\varepsilon > 0$, we have

$$\mathbb{P}\left(\left|\frac{1}{n}\sum_{i=1}^{n} X_i\right| > \varepsilon\right) \leq \frac{\sigma^2}{n\varepsilon^2}.$$

Prompt

Create a file `Chebyshev.lean` in the `LearnLean` directory.
In this file, state and prove the following result.

`<paste LaTeX code here...>`

Getting help

- `#check`: Check the type of an expression.
  Propositions and proofs have types and can be checked!
  For example, `#check Nat.eq_one_of_dvd_one`

Getting help

- `#check`: Check the type of an expression.
  Propositions and proofs have types and can be checked!
  For example, `#check Nat.eq_one_of_dvd_one`
- `#eval`: Evaluate some Lean code.
  For example, `#eval Nat.minFac 1`

Getting help

- `#check`: Check the type of an expression.
  Propositions and proofs have types and can be checked!
  For example, `#check Nat.eq_one_of_dvd_one`
- `#eval`: Evaluate some Lean code.
  For example, `#eval Nat.minFac 1`
- Advanced tactics for finding proofs:
  `aesop`, `omega`, `linarith`, `ring`, `grind`

Getting help

- `#check`: Check the type of an expression.
  Propositions and proofs have types and can be checked!
  For example, `#check Nat.eq_one_of_dvd_one`
- `#eval`: Evaluate some Lean code.
  For example, `#eval Nat.minFac 1`
- Advanced tactics for finding proofs:
  `aesop`, `omega`, `linarith`, `ring`, `grind`
- `https://leansearch.net/`:
  Search Mathlib using natural language.

Getting help

- `#check`: Check the type of an expression.
  Propositions and proofs have types and can be checked!
  For example, `#check Nat.eq_one_of_dvd_one`
- `#eval`: Evaluate some Lean code.
  For example, `#eval Nat.minFac 1`
- Advanced tactics for finding proofs:
  `aesop`, `omega`, `linarith`, `ring`, `grind`
- `https://leansearch.net/`:
  Search Mathlib using natural language.
- `github.com/leanprover-community/mathlib4`
  The Mathlib code repository.

Getting help

- `#check`: Check the type of an expression.
  Propositions and proofs have types and can be checked!
  For example, `#check Nat.eq_one_of_dvd_one`
- `#eval`: Evaluate some Lean code.
  For example, `#eval Nat.minFac 1`
- Advanced tactics for finding proofs:
  `aesop`, `omega`, `linarith`, `ring`, `grind`
- `https://leansearch.net/`:
  Search Mathlib using natural language.
- `github.com/leanprover-community/mathlib4`
  The Mathlib code repository.
- `https://lean-lang.org/`: Core documentation for Lean.